BATMAN ÜNİVERSİTESİ

SAYISAL GÖRÜNTÜ İŞLEME DERSİ PROJESİ

CHEST X-RAY GÖRÜNTÜLERINDE HISTOGRAM TABANLI KONTRAST GELIŞTIRME VE PNEUMONIA(ZATÜRRE) TESPITI

ÖĞRENCİNİN

ADI SOYADI : EMRAH TUSUN

DERSI : Sayısal Görüntü İşleme

NUMARASI : 220205011033

İçindekiler

Özet 3
1. Giriş 3
2. Veri Seti 3
3. Sistem Mimarisi 4
3.1 Genel Mimari 4
3.2 Frontend (Flutter) 4
3.3 Backend (Python – Flask) 4
4. Kullanılan Yöntemler 5
4.1 Histogram Eşitleme 5
4.2 CLAHE (Contrast Limited Adaptive Histogram Equalization) 5
5. Görüntü Kalite Metrikleri 5
5.1 PSNR (Peak Signal-to-Noise Ratio) 5
5.2 SSIM (Structural Similarity Index) 5
6. Kodların Açıklanması 5
6.1 Backend – Flask Sunucu (main.py) 6
6.2 Görüntü İşleme Fonksiyonları (image_processing.py) 10
6.3 Yapay zeka modelinin eğitimi (train_model.py) 11
6.4 Yapay Zeka Modelinin TFLite'a Dönüştürülmesi (convert_to_tflite.py) 15
6.5 Frontend – Flutter Mobil Uygulama (main.dart) 17
7. Yapay Zeka Entegrasyonu 20
8. Deneysel Sonuçlar 20
9. Sonuç ve Değerlendirme 21
10. Kaynakça 21

Özet

Bu çalışmada, akciğer röntgeni (Chest X-Ray) görüntüleri üzerinde histogram tabanlı kontrast iyileştirme yöntemleri uygulanmış ve elde edilen sonuçlar nicel metrikler ile değerlendirilmiştir. Düşük kontrast problemi, tıbbi görüntülerde hem görsel yorumlamayı hem de makine öğrenmesi tabanlı otomatik analiz sistemlerini olumsuz etkilemektedir. Bu kapsamda klasik histogram eşitleme ve Contrast Limited Adaptive Histogram Equalization (CLAHE) yöntemleri kullanılarak görüntü kalitesi artırılmıştır.

Uygulanan yöntemlerin başarımı Peak Signal-to-Noise Ratio (PSNR) ve Structural Similarity Index (SSIM) metrikleri ile ölçülmüştür. Ayrıca kontrast iyileştirmenin, TensorFlow Lite tabanlı bir pneumonia sınıflandırma modeli üzerindeki etkisi analiz edilmiştir. Geliştirilen sistem, Flutter tabanlı bir mobil uygulama ve Python–Flask tabanlı bir backend mimarisi kullanılarak uçtan uca bir yapı halinde tasarlanmıştır. Elde edilen sonuçlar, CLAHE yönteminin tıbbi görüntülerde daha dengeli ve yapısal bütünlüğü koruyan sonuçlar ürettiğini göstermektedir.

1. Giriş

Sayısal görüntü işleme, özellikle tıbbi görüntüleme alanında tanı destek sistemlerinin geliştirilmesinde kritik bir rol oynamaktadır. Akciğer röntgeni (Chest X-Ray) görüntüleri, zatürre (pneumonia) gibi hastalıkların tespitinde yaygın olarak kullanılmakta; ancak bu görüntüler çoğu zaman düşük kontrast, aydınlatma düzensizliği ve gürültü gibi problemler içermektedir. Bu durum hem insan gözüyle yapılan değerlendirmeyi hem de makine öğrenmesi tabanlı otomatik analizleri olumsuz etkilemektedir.

Bu projede, Chest X-Ray görüntüleri üzerinde histogram analizi yapılarak kontrast iyileştirme yöntemleri uygulanmıştır. Klasik histogram eşitleme (Histogram Equalization) ve tıbbi görüntülerde daha başarılı sonuçlar veren CLAHE (Contrast Limited Adaptive Histogram Equalization) yöntemleri karşılaştırmalı olarak incelenmiştir. Elde edilen sonuçlar PSNR ve SSIM metrikleri ile nicel olarak değerlendirilmiş, ayrıca kontrast artırmanın derin öğrenme tabanlı bir sınıflandırma modeli üzerindeki etkisi gözlemlenmiştir.

2. Veri Seti

Bu çalışmada Kaggle platformunda yayımlanan **Chest X-Ray Dataset (Pneumonia)** veri seti kullanılmıştır. Veri seti, normal ve pneumonia sınıflarına ait çok sayıda akciğer röntgeni görüntüsü içermektedir. Görüntüler gri-seviye formatında olup, histogram tabanlı kontrast iyileştirme yöntemlerinin uygulanmasına uygundur.

Veri setinin tıbbi nitelikte olması nedeniyle, görüntülerin yapısal bütünlüğünün korunması büyük önem taşımaktadır. Bu nedenle kontrast artırma işlemleri yalnızca görsel iyileştirme açısından değil, aynı zamanda yapısal benzerlik metrikleri açısından da değerlendirilmiştir.

3. Sistem Mimarisi

Geliştirilen sistem, istemci–sunucu mimarisi temel alınarak tasarlanmıştır. Mobil istemci tarafında Flutter kullanılırken, sunucu tarafında Python tabanlı Flask framework'ü tercih edilmiştir. Görüntü işleme ve değerlendirme işlemleri backend üzerinde gerçekleştirilmiş, sonuçlar RESTful API aracılığıyla mobil uygulamaya iletilmiştir.

3.1 Genel Mimari

Sistemin genel çalışma akışı aşağıdaki gibidir:

- Kullanıcı röntgen görüntüsünü seçer.
- Görüntü backend servisine gönderilir.
- Histogram eşitleme ve CLAHE uygulanır.
- PSNR ve SSIM metrikleri hesaplanır.
- Yapay zeka modeli ile tahmin yapılır.
- Sonuçlar mobil uygulamada görsel olarak sunulur.

Backend servisi olarak **PythonAnywhere** platformu üzerinde yayınlanmıştır. Bu sayede sistem, gerçek zamanlı ve platform bağımsız şekilde çalışabilmektedir.

3.2 Frontend (Flutter)

Flutter, tek kod tabanı ile mobil platformlarda çalışabilen modern bir UI framework'üdür. Bu projede Flutter, kullanıcıdan görüntü alma (kamera veya galeri), backend servisine görüntü gönderme ve elde edilen analiz sonuçlarını görsel olarak sunma amacıyla kullanılmıştır.

Mobil uygulama, kullanıcı etkileşimini minimumda tutacak şekilde tasarlanmış; analiz süreci otomatik olarak başlatılarak sonuçlar tek bir ekran üzerinden karşılaştırmalı biçimde sunulmuştur.

3.3 Backend (Python – Flask)

Backend tarafında Flask framework'ü kullanılarak REST API geliştirilmiştir. Flask, hafif ve modüler yapısı sayesinde PythonAnywhere gibi kısıtlı ortamlarda verimli şekilde çalışabilmektedir.

Backend; **OpenCV** ve **NumPy** kütüphaneleri ile görüntü işleme işlemlerini gerçekleştirmekte, PSNR ve SSIM hesaplamalarını yapmakta ve sonuçları JSON formatında istemciye iletmektedir.

4. Kullanılan Yöntemler

4.1 Histogram Eşitleme

Histogram eşitleme, bir görüntünün yoğunluk dağılımını yeniden düzenleyerek kontrastı artırmayı amaçlayan klasik bir yöntemdir. Bu yöntemde, görüntünün histogramı daha uniform bir dağılıma dönüştürülür. OpenCV kütüphanesinin **equalizeHist** fonksiyonu kullanılarak uygulama gerçekleştirilmiştir.

Histogram eşitleme, global bir yöntem olduğu için bazı durumlarda görüntünün belirli bölgelerinde aşırı kontrast oluşmasına neden olabilmektedir.

4.2 CLAHE (Contrast Limited Adaptive Histogram Equalization)

CLAHE yöntemi, görüntüyü küçük bölgelere ayırarak her bölge için ayrı histogram eşitleme uygular. Ayrıca kontrast sınırlandırması (clip limit) sayesinde gürültünün aşırı büyümesi engellenir. Bu özellikleri nedeniyle CLAHE, tıbbi görüntülerde daha dengeli ve güvenilir sonuçlar üretmektedir.

Bu projede CLAHE parametreleri, görsel kaliteyi koruyacak şekilde seçilmiştir (clipLimit=2.0, tileGridSize=(8,8)).

5. Görüntü Kalite Metrikleri

5.1 PSNR (Peak Signal-to-Noise Ratio)

PSNR, orijinal görüntü ile işlenmiş görüntü arasındaki farkı ölçen bir metriktir. Değerin yüksek olması, işleme sonrası görüntünün orijinale daha yakın olduğunu ifade eder. Bu projede PSNR, ortalama karesel hata (MSE) üzerinden hesaplanmıştır.

5.2 SSIM (Structural Similarity Index)

SSIM, görüntülerin yapısal benzerliğini ölçen bir metriktir. İnsan görme sistemini modellemeye çalıştığı için PSNR'a göre daha anlamlı sonuçlar sunar. SSIM değerinin 1'e yakın olması, yapısal bütünlüğün korunduğunu gösterir.

Bu çalışmada, hesaplama karmaşıklığını azaltmak amacıyla OpenCV'nin benzerlik ölçüm fonksiyonları kullanılarak basitleştirilmiş bir SSIM yaklaşımı uygulanmıştır.

6. Kodların Açıklanması

Bu bölümde projede kullanılan yazılım bileşenleri ve ilgili kod blokları detaylı şekilde açıklanmıştır. Kodlar, modüler ve okunabilir bir yapı gözetilerek geliştirilmiştir.

6.1 Backend – Flask Sunucu (main.py)

Kod Bloğu 1: Kütüphanelerin İçe Aktarılması

```
import os
import cv2
import numpy as np
import base64
from flask import Flask, request, jsonify
from flask_cors import CORS
import tflite_runtime.interpreter as tflite
from image_processing import apply_hist_eq, apply_clahe
```

Bu kod bloğunda backend uygulamasında kullanılacak temel kütüphaneler projeye dahil edilmiştir. OpenCV (cv2) ve NumPy (numpy) görüntü işleme ve matematiksel hesaplamalar için kullanılırken, Flask framework'ü HTTP tabanlı bir web servisi geliştirmek amacıyla tercih edilmiştir. flask_cors modülü, mobil istemciden gelen isteklerde CORS (Cross-Origin Resource Sharing) problemlerini önlemek için kullanılmıştır.
Ayrıca, TensorFlow Lite yorumlayıcısı ile eğitilmiş yapay zeka modelinin backend üzerinde çalıştırılması sağlanmıştır. Görüntü işleme fonksiyonları modülerlik amacıyla ayrı bir dosyadan içe aktarılmıştır.

Kod Bloğu 2: Flask Uygulamasının Oluşturulması ve CORS Ayarı

```

app = Flask(__name__)
CORS(app)

```

Bu bölümde Flask uygulaması oluşturulmuş ve CORS desteği aktif hale getirilmiştir. Bu sayede Flutter tabanlı mobil uygulama, farklı bir domain üzerinden backend servisine sorunsuz şekilde istek gönderebilmektedir. Bu yapı, istemci–sunucu mimarisinin sağlıklı çalışması için gereklidir.

Kod Bloğu 3: Model Dosya Yolu ve TFLite Yorumlayıcısının Yüklenmesi

Bu kod bloğunda, PythonAnywhere üzerinde barındırılan TensorFlow Lite modelinin dosya yolu tanımlanmıştır. Model, uygulama başlatılırken belleğe yüklenmekte ve gerekli tensörler ayrılmaktadır. Bu yaklaşım, her istek sırasında modeli yeniden yükleme ihtiyacını ortadan kaldırarak performans artışı sağlamaktadır.
Modelin giriş ve çıkış tensör bilgileri alınarak, ilerleyen aşamalarda yapılacak tahmin işlemleri için hazır hale getirilmiştir.

```python
BASE_DIR = "/home/hsherlock/mysite"
MODEL_PATH = os.path.join(BASE_DIR, "pneumonia_model.tflite")

interpreter = tflite.Interpreter(model_path=MODEL_PATH)
interpreter.allocate_tensors()
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
```

Kod Bloğu 4: PSNR ve SSIM Hesaplama Fonksiyonu

```python
def calculate_metrics_simple(original, processed):
    mse = np.mean((original - processed) ** 2)
    if mse == 0:
        p_val = 100
    else:
        p_val = round(20 * np.log10(255.0 / np.sqrt(mse)), 2)

    res = cv2.matchTemplate(original, processed, cv2.TM_CCOEFF_NORMED)[0][0]
    s_val = round(float(res), 4)

    return p_val, s_val
```

Bu fonksiyon, orijinal görüntü ile işlenmiş görüntü arasındaki kalite farkını ölçmek amacıyla PSNR ve SSIM metriklerini hesaplamaktadır. PSNR değeri, ortalama karesel hata (MSE) üzerinden hesaplanmakta ve görüntü bozulma seviyesini ifade etmektedir.
SSIM hesaplaması ise hesaplama maliyetini düşürmek amacıyla OpenCV'nin normalize edilmiş benzerlik fonksiyonu kullanılarak basitleştirilmiş şekilde uygulanmıştır. Bu metrikler, kontrast iyileştirme yöntemlerinin görüntü yapısını ne ölçüde koruduğunu analiz etmek için kullanılmıştır.

Kod Bloğu 5: TFLite Modeli ile Tahmin Fonksiyonu

Bu fonksiyon, TensorFlow Lite formatındaki pneumonia sınıflandırma modelini kullanarak görüntü üzerinde tahmin yapmaktadır. Görüntü, modelin beklediği giriş boyutuna yeniden ölçeklendirilmekte ve normalize edilmektedir.
Model çıktısı yüzde cinsinden ifade edilerek, kontrast iyileştirme işlemlerinin yapay zeka tahminine etkisi karşılaştırmalı olarak analiz edilebilmektedir.

```python
def predict_tflite(img):
    img_res = cv2.resize(img, (224, 224))
    if len(img_res.shape) == 2:
        img_res = cv2.cvtColor(img_res, cv2.COLOR_GRAY2RGB)
    img_array = np.expand_dims(img_res / 255.0, axis=0).astype(np.float32)
    interpreter.set_tensor(input_details[0]['index'], img_array)
    interpreter.invoke()
    prediction = interpreter.get_tensor(output_details[0]['index'])[0][0]
    return round(float(prediction) * 100, 2)
```

Kod Bloğu 6: Görüntünün Base64 Formatına Dönüştürülmesi

```python
def encode_img_to_base64(img):
    _, buffer = cv2.imencode('.jpg', img)
    return base64.b64encode(buffer).decode('utf-8')
```

Bu fonksiyon, backend üzerinde işlenen görüntülerin mobil uygulamaya geri gönderilebilmesi için Base64 formatına dönüştürülmesini sağlamaktadır. Bu yöntem sayesinde görüntüler JSON veri yapısı içerisinde güvenli ve uyumlu bir şekilde iletilebilmektedir.

Kod Bloğu 7 – API Endpoint Tanımı

```python
@app.route('/analyze', methods=['POST'])
def analyze():
```

Bu blok, Flask framework'ü kullanılarak oluşturulan RESTful API uç noktasını tanımlar. /analyze endpoint'i, istemciden HTTP POST yöntemi ile gelen istekleri karşılamak üzere yapılandırılmıştır. Bu endpoint'in temel amacı, gönderilen akciğer X-Ray görüntüsünü alarak hem görüntü işleme hem de makine öğrenmesi tabanlı analizleri gerçekleştirmektir. Endpoint, tüm analiz sürecini tek bir istek–yanıt döngüsü içerisinde yürütmektedir.

Kod Bloğu 8 – Görüntünün Alınması ve Decode Edilmesi

Bu bölümde istemciden gönderilen görüntü dosyası request.files üzerinden okunmaktadır. Alınan byte verisi NumPy dizisine dönüştürülerek OpenCV'nin imdecode fonksiyonu ile görüntü haline getirilir. Ardından renkli (BGR) görüntü, kontrast iyileştirme işlemlerinde kullanılmak üzere gri seviye (grayscale) formata dönüştürülür. Bu adım, histogram tabanlı işlemler için zorunludur.

```
file = request.files['image'].read()
npimg = np.frombuffer(file, np.uint8)
img = cv2.imdecode(npimg, cv2.IMREAD_COLOR)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Kod Bloğu 9 – Orijinal Görüntü Üzerinde Model Tahmini

```
res_orig = predict_tflite(img)
```

Bu satırda, herhangi bir görüntü iyileştirme uygulanmadan, orijinal X-Ray görüntüsü TensorFlow Lite tabanlı derin öğrenme modeli ile analiz edilmektedir. Model çıktısı, görüntüde pneumonia (zatürre) bulunma olasılığını yüzde (%) cinsinden ifade etmektedir. Bu değer, karşılaştırma amacıyla referans skor olarak kullanılmaktadır.

Kod Bloğu 10 – Histogram Eşitleme (Histogram Equalization)

```
hist_img = apply_hist_eq(gray)
h_psnr, h_ssim = calculate_metrics_simple(gray, hist_img)
res_hist = predict_tflite(cv2.cvtColor(hist_img, cv2.COLOR_GRAY2RGB))
```

Bu blokta klasik histogram eşitleme yöntemi uygulanmaktadır. Histogram eşitleme, gri seviye görüntünün kontrastını artırarak detayların daha belirgin hale gelmesini sağlar. İşlem sonrasında, elde edilen görüntü ile orijinal görüntü arasındaki kalite farkı PSNR ve SSIM metrikleri kullanılarak hesaplanmaktadır. Son olarak histogram eşitlenmiş görüntü, modelin beklediği RGB formatına dönüştürülerek tekrar sınıflandırmaya tabi tutulur.

Kod Bloğu 11 – CLAHE (Contrast Limited Adaptive Histogram Equalization)

```
clahe_img = apply_clahe(gray)
c_psnr, c_ssim = calculate_metrics_simple(gray, clahe_img)
res_clahe = predict_tflite(cv2.cvtColor(clahe_img, cv2.COLOR_GRAY2RGB))
```

Bu bölümde adaptif kontrast iyileştirme yöntemi olan CLAHE uygulanmaktadır. CLAHE, görüntüyü küçük bölgelere ayırarak her bölge için ayrı histogram eşitleme yapar ve aşırı kontrast artışını sınırlar. Bu sayede gürültü artışı minimize edilir. CLAHE uygulanmış görüntü için de PSNR ve SSIM değerleri hesaplanmakta ve model tahmini alınmaktadır.

Kod Bloğu 12 – Sonuçların JSON Formatında Döndürülmesi

Bu son blokta tüm analiz sonuçları JSON formatında istemciye geri döndürülmektedir. Orijinal, histogram eşitleme uygulanmış ve CLAHE uygulanmış görüntüler için pneumonia tahmin skorları ile birlikte PSNR ve SSIM metrikleri sunulmaktadır. Ayrıca işlenmiş görüntüler Base64 formatına dönüştürülerek frontend tarafında doğrudan görüntülenebilir hale getirilmektedir.

```
return jsonify({
    "original_score": res_orig,
    "hist_score": res_hist,
    "hist_psnr": h_psnr,
    "hist_ssim": h_ssim,
    "clahe_score": res_clahe,
    "clahe_psnr": c_psnr,
    "clahe_ssim": c_ssim,
    "hist_img_base64": encode_img_to_base64(hist_img),
    "clahe_img_base64": encode_img_to_base64(clahe_img)
})
```

6.2 Görüntü İşleme Fonksiyonları (image_processing.py)

Bu dosya, kontrast iyileştirme işlemlerini gerçekleştiren yardımcı fonksiyonları içermektedir. Modüler yapı sayesinde ana sunucu kodu sade tutulmuştur.

```
import cv2

def apply_hist_eq(image_gray):
    return cv2.equalizeHist(image_gray)

def apply_clahe(image_gray):
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
    return clahe.apply(image_gray)
```

import cv2: görüntü işleme işlemlerinin gerçekleştirilmesi için OpenCV kütüphanesini projeye dahil eder. OpenCV, piksel tabanlı dönüşümler, histogram işlemleri ve kontrast iyileştirme gibi temel görüntü işleme fonksiyonlarını sağlamaktadır.

apply_hist_eq: Bu fonksiyon, giriş olarak verilen gri seviye görüntüye klasik histogram eşitleme işlemi uygular. Histogram eşitleme, görüntüdeki piksel yoğunluklarını yeniden dağıtarak kontrastın artırılmasını sağlar. Özellikle düşük kontrastlı X-Ray görüntülerinde detayların daha belirgin hale gelmesine katkıda bulunur. Fonksiyon, kontrastı iyileştirilmiş görüntüyü çıktı olarak döndürür.

apply_clahe: Bu fonksiyon, CLAHE (Contrast Limited Adaptive Histogram Equalization) algoritmasını uygular. Görüntü küçük parçalara bölünerek her parça için ayrı histogram eşitleme yapılır. clipLimit parametresi aşırı kontrast artışını sınırlandırırken, tileGridSize parametresi bölge boyutunu belirler.

6.3 Yapay zeka modelinin eğitimi (train_model.py)

TensorFlow ve Keras Kütüphanelerinin İçe Aktarılması

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
import os
```

Veri ve Model Parametrelerinin Tanımlanması

```
DATA_PATH = '../data/train'
MODEL_SAVE_PATH = 'pneumonia_model.h5'
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
EPOCHS = 10
```

Bu satırlarda model eğitimi için gerekli temel parametreler tanımlanmaktadır. Eğitim verilerinin bulunduğu dizin yolu, eğitilen modelin kaydedileceği dosya adı, giriş görüntülerinin boyutu, batch büyüklüğü ve eğitim süresince kullanılacak epoch sayısı belirlenmektedir.

Model Eğitim Fonksiyonunun Tanımlanması

```
def train_my_model():
```

Bu fonksiyon, pneumonia tespiti için kullanılacak derin öğrenme modelinin baştan sona eğitim sürecini kapsayan ana fonksiyondur. Tüm veri hazırlama, model oluşturma, eğitim ve kaydetme işlemleri bu fonksiyon içerisinde yürütülmektedir.

Veri Artırma (Data Augmentation) Tanımı

Bu blokta ImageDataGenerator kullanılarak veri artırma işlemleri tanımlanmaktadır. Görüntüler normalize edilmekte, rastgele döndürme, yakınlaştırma ve yatay çevirme uygulanmaktadır. Ayrıca verinin %20'si doğrulama (validation) seti olarak ayrılmaktadır. Bu yöntem, modelin genelleme yeteneğini artırmak için kullanılır.

```
    train_datagen = ImageDataGenerator(
        rescale=1./255,
        rotation_range=20,
        zoom_range=0.2,
        horizontal_flip=True,
        validation_split=0.2
    )
```

Eğitim Veri Kümesinin Oluşturulması

```
    train_generator = train_datagen.flow_from_directory(
        DATA_PATH,
        target_size=IMG_SIZE,
        batch_size=BATCH_SIZE,
        class_mode='binary',
        subset='training'
    )
```

Bu bölümde eğitim verileri, dizin yapısına göre otomatik olarak okunmaktadır. Görüntüler belirtilen boyuta ölçeklenmekte ve binary sınıflandırma (pneumonia / normal) problemi için etiketlenmektedir. Bu generator yalnızca eğitim verilerini içermektedir.

Doğrulama (Validation) Veri Kümesinin Oluşturulması

```
val_generator = train_datagen.flow_from_directory(
    DATA_PATH,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='validation'
)
```

Bu blok, modelin eğitim sırasında performansını değerlendirmek için kullanılan doğrulama veri kümesini oluşturmaktadır. Eğitim verilerinden ayrılan %20'lik kısım validation seti olarak kullanılmaktadır.

Önceden Eğitilmiş MobileNetV2 Modelinin Yüklenmesi

```
base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
```

Bu satırda ImageNet veri seti üzerinde önceden eğitilmiş MobileNetV2 modeli yüklenmektedir. include_top=False parametresi, modelin son sınıflandırma katmanlarının çıkarılmasını sağlar. Bu yaklaşım transfer learning tekniği olarak adlandırılır.

Base Model Katmanlarının Dondurulması

```
for layer in base_model.layers:
    layer.trainable = False
```

Bu döngü ile MobileNetV2 modelinin tüm katmanları eğitime kapatılmaktadır. Böylece önceden öğrenilmiş özellikler korunur ve yalnızca eklenen yeni katmanlar eğitilir.

Özel Sınıflandırma Katmanlarının Eklenmesi

```
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x)
predictions = Dense(1, activation='sigmoid')(x)
```

Bu bölümde base modelin çıkışına yeni katmanlar eklenmektedir. GlobalAveragePooling2D

ile özellik haritaları özetlenir, ardından ReLU aktivasyonlu tam bağlantılı katman eklenir. Son katman sigmoid aktivasyonu ile pneumonia olasılığını üretir.

Modelin Oluşturulması ve Derlenmesi (Compile)

```
model = Model(inputs=base_model.input, outputs=predictions)

model.compile(optimizer=Adam(learning_rate=0.001), loss='binary_crossentropy', metrics=['accuracy'])
```

İlk satırda giriş olarak MobileNetV2'nin girdisi, çıkış olarak ise oluşturulan sınıflandırma katmanı alınarak nihai model oluşturulmaktadır.
Model, Adam optimizasyon algoritması kullanılarak derlenmektedir. Binary sınıflandırma problemi olduğu için kayıp fonksiyonu olarak binary cross-entropy seçilmiştir. Performans metriği olarak doğruluk (accuracy) kullanılmaktadır.

Modelin Eğitilmesi

```
model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=EPOCHS
)
```

Bu blokta model, belirlenen epoch sayısı boyunca eğitim verileri ile eğitilmektedir. Eğitim sırasında doğrulama verileri kullanılarak modelin performansı izlenmektedir.

Modelin Kaydedilmesi

```
model.save(MODEL_SAVE_PATH)
print(f"Eğitim tamamlandı! Model '{MODEL_SAVE_PATH}' olarak kaydedildi.")
```

Eğitim tamamlandıktan sonra model .h5 formatında diske kaydedilmektedir. Bu model daha sonra backend tarafında TensorFlow Lite formatına dönüştürülerek kullanılmaktadır.

Ana Program Kontrolü

Bu yapı, dosya doğrudan çalıştırıldığında model eğitim fonksiyonunun otomatik olarak başlatılmasını sağlar.

```
if __name__ == "__main__":
    train_my_model()
```

6.4 Yapay Zeka Modelinin TFLite'a Dönüştürülmesi (convert_to_tflite.py)

Bu dosya, Keras formatında eğitilmiş olan pneumonia sınıflandırma modelinin TensorFlow Lite formatına dönüştürülmesi amacıyla kullanılmıştır.

---

1. Gerekli Kütüphanelerin Yüklenmesi

Bu bölümde TensorFlow kütüphanesi derin öğrenme modeli işlemleri için, os kütüphanesi ise dosya varlığını kontrol etmek amacıyla projeye dahil edilmiştir.

```
import tensorflow as tf
import os
```

---

2. Girdi ve Çıktı Model Dosyalarının Tanımlanması

```
model_input = 'pneumonia_model.h5'
model_output = 'pneumonia_model.tflite'
```

Eğitilmiş Keras modeli (.h5) giriş olarak tanımlanmış, backend ortamında kullanılmak üzere dönüştürülecek TensorFlow Lite modeli (.tflite) çıktı dosyası olarak belirlenmiştir.

---

3. Model Dosyasının Varlık Kontrolü

```
if not os.path.exists(model_input):
    print(f"HATA: {model_input} hala bulunamadı!")
else:
```

Dönüştürme işlemine başlamadan önce eğitilmiş model dosyasının sistemde mevcut olup olmadığı kontrol edilmektedir. Dosya bulunamazsa işlem güvenli şekilde durdurulur.

---

4. Keras Modelinin Yüklenmesi

```
model = tf.keras.models.load_model(model_input)
```

Daha önce eğitilmiş ve kaydedilmiş olan pneumonia tespit modeli belleğe yüklenir. Bu model, TensorFlow Lite formatına dönüştürülecek ana yapıdır.

---

5. TensorFlow Lite Dönüştürücüsünün Oluşturulması

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
```

Yüklenen Keras modeli, mobil ve düşük kaynaklı ortamlarda çalışabilmesi için TensorFlow Lite dönüştürücüsüne aktarılır.

---

6. PythonAnywhere Uyumluluk Ayarları

```
#PythonAnywhere uyumluluğu çin
converter.target_spec.supported_ops = [
    tf.lite.OpsSet.TFLITE_BUILTINS,
    tf.lite.OpsSet.SELECT_TF_OPS
]
```

Bu ayar, PythonAnywhere gibi kısıtlı ortamlarda modelin sorunsuz çalışabilmesi için gerekli olan işlem (operation) desteğini genişletir. Böylece modelin dönüştürme ve çalışma sırasında hata vermesi engellenir.

---

7. Modelin TFLite Formatına Dönüştürülmesi , TFLite Modelinin Dosyaya Kaydedilmesi ve İşlem Sonu Bilgilendirmesi

```
tflite_model = converter.convert()
with open(model_output, 'wb') as f:
    f.write(tflite_model)
print(f"BAŞARILI! Yeni dosya: {model_output}")
```

6.5 Frontend – Flutter Mobil Uygulama (main.dart)

Flutter tarafında geliştirilen mobil uygulama, kullanıcıdan röntgen görüntüsünü alma, backend servisine gönderme ve analiz sonuçlarını görsel olarak sunma görevlerini üstlenmektedir.

---

1. Kütüphanelerin İçe Aktarılması

```
import 'dart:convert';
import 'dart:io';
import 'dart:typed_data';
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';
import 'package:http/http.dart' as http;
```

Bu bölümde Flutter uygulamasında kullanılacak temel kütüphaneler tanımlanmıştır. Görüntü seçimi (image_picker), HTTP üzerinden sunucuya veri gönderimi (http) ve base64 kodlama/çözme işlemleri için gerekli paketler projeye dahil edilmiştir.

---

2. Uygulamanın Başlatılması

```
void main() {
  runApp(const MaterialApp(
    home: HomePage(),
    debugShowCheckedModeBanner: false,
  ));
}
```

Flutter uygulamasının giriş noktasıdır. Uygulama başlatıldığında ana ekran olarak HomePage widget'ı yüklenir.

---

3. Ana Sayfa (HomePage) Yapısı

```
class HomePage extends StatefulWidget {
  const HomePage({super.key});
  @override
  State<HomePage> createState() => _HomePageState();
}
```

Kullanıcının röntgen görüntüsünü galeriden veya kameradan seçebildiği ana arayüzdür. Stateful yapı, kullanıcı etkileşimlerine bağlı olarak ekranın güncellenmesini sağlar

---

4. Görüntü Seçme İşlemi

```
Future<void> _pickImage(ImageSource source) async {
  try {
    final XFile? image = await _picker.pickImage(
      source: source,
      maxWidth: 1800,
      maxHeight: 1800,
    );
```

Bu fonksiyon, kullanıcının galeriden veya kameradan bir X-Ray görüntüsü seçmesini sağlar. Görüntü seçildikten sonra analiz ekranına yönlendirme yapılır.

---

5. Analiz Sayfasına Geçiş

```
Navigator.push(
    context,
    MaterialPageRoute(builder: (context) => AnalysisPage(imageFile: File(image.path)))
);
```

Seçilen görüntü dosyası analiz sayfasına parametre olarak aktarılır. Böylece görüntü backend'e gönderilmek üzere hazır hale getirilir.

---

6. AnalysisPage ve Otomatik Analiz Başlatma

```
@override
void initState() {
  super.initState();
  _analyze();
}
```

Analiz sayfası açıldığında _analyze() fonksiyonu otomatik olarak çalıştırılır ve görüntü sunucuya gönderilir.

---

7. Backend'e Görüntü Gönderimi

```
var req = http.MultipartRequest('POST', Uri.parse('https://hsherlock.pythonanywhere.com/analyze'));
```

Seçilen röntgen görüntüsü HTTP POST isteği ile Flask tabanlı backend servisine gönderilir. Backend, görüntü üzerinde histogram ve CLAHE işlemlerini uygular.

---

8. Sunucudan Gelen Sonuçların Alınması

```
setState(() => _res = json.decode(res.body));
```

Backend tarafından dönen JSON verisi çözülerek uygulama içerisinde kullanılmak üzere saklanır. Bu veri PSNR, SSIM ve yapay zekâ tahmin sonuçlarını içerir.

---

9. Sonuçların Kart Yapısında Gösterimi

```
Widget _buildCard(String title, Widget imgWidget, dynamic score, dynamic psnr, dynamic ssim, Color headerColor) {
  return Card(
    elevation: 3,
    margin: const EdgeInsets.only(bottom: 15),
    clipBehavior: Clip.antiAlias,
    child: Column(children: [
      Container(
          color: headerColor,
          width: double.infinity,
          padding: const EdgeInsets.all(8),
          child: Text(title, style: const TextStyle(color: Colors.white, fontWeight: FontWeight.bold))
      ),
      Container(color: Colors.black, child: imgWidget),
      Padding(
        padding: const EdgeInsets.all(15),
        child: Row(mainAxisAlignment: MainAxisAlignment.spaceAround, children: [
          _metricItem("PSNR", "$psnr dB"),
          _metricItem("SSIM", "$ssim"),
          _metricItem("AI Tahmini", "%$score"),
        ]),
```

Orijinal görüntü, histogram eşitleme sonucu ve CLAHE sonucu ayrı kartlar halinde kullanıcıya sunulur. Her kartta görüntü, PSNR, SSIM ve AI tahmin yüzdesi birlikte gösterilir.

---

10. Teknik Bilgi Paneli

```
Widget _buildInfoSection() {
  return Container(
    padding: const EdgeInsets.all(15),
    decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(10),
        border: Border.all(color: Colors.blue.withOpacity(0.2))
    ),
    child: Column(crossAxisAlignment: CrossAxisAlignment.start, children: [
      const Text("Teknik Analiz Rehberi", style: TextStyle(fontWeight: FontWeight.bold, fontSize: 16, color: Colors.blue)),
      const Divider(),
      _infoText("• PSNR (dB): Görüntü kalitesini ölçer. Değer ne kadar yüksekse, işlem sonrası görüntü o kadar az bozulmuştur."),
      _infoText("• SSIM: Yapısal benzerlik endeksidir. 1'e yakın olması, kemik ve doku yapısının orijinalle aynı kaldığını ispatlar."),
      _infoText("• Tahmin (%): Derin öğrenme modelinin ilgili görüntü üzerinde saptadığı Pneumonia (Zatürre) olasılığıdır."),
    ]),
  );
}
```

Bu bölümde PSNR, SSIM ve yapay zekâ tahmin skorlarının ne anlama geldiği kullanıcıya kısa ve anlaşılır şekilde açıklanmaktadır.

7. Yapay Zeka Entegrasyonu

Projede kontrast artırma işlemlerinin, derin öğrenme tabanlı bir pneumonia sınıflandırma modeli üzerindeki etkisi de gözlemlenmiştir. **Keras** formatında eğitilmiş olan model, PythonAnywhere ortamının kısıtları nedeniyle **TensorFlow Lite (TFLite)** formatına dönüştürülmüştür.

TFLite modeli, kontrastı artırılmış görüntüler üzerinde çalıştırılarak elde edilen tahmin sonuçları karşılaştırılmıştır. Bu sayede görüntü ön işleme adımlarının yapay zeka performansına etkisi analiz edilmiştir.

8. Deneysel Sonuçlar

Bu bölümde orijinal görüntü, histogram eşitleme ve CLAHE sonuçları karşılaştırmalı olarak sunulmuştur. Elde edilen PSNR ve SSIM değerleri, CLAHE yönteminin genel olarak daha dengeli ve yapısal olarak daha başarılı sonuçlar verdiğini göstermektedir.

Normal

zatürre (pneumonia)

9. Sonuç ve Değerlendirme

Bu projede, Chest X-Ray görüntüleri üzerinde histogram tabanlı kontrast iyileştirme yöntemleri uygulanmış ve sonuçlar nicel metriklerle değerlendirilmiştir. CLAHE yönteminin, klasik histogram eşitlemeye kıyasla tıbbi görüntülerde daha uygun olduğu gözlemlenmiştir.

Ayrıca, kontrast artırma işlemlerinin derin öğrenme tabanlı sınıflandırma performansını olumlu yönde etkileyebileceği görülmüştür. Gelecek çalışmalarda farklı kontrast iyileştirme tekniklerinin ve daha gelişmiş kalite metriklerinin kullanılması planlanmaktadır.

10. Kaynakça

[1] P. T. Mooney, "Chest X-Ray Images (Pneumonia)," *Kaggle Dataset*, 2018. [Çevrimiçi].
Erişim adresi: https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia

[2] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson Education, 2018.

[3] Z. Wang, A. C. Bovik, H. R. Sheikh ve E. P. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE Transactions on Image Processing*, cilt 13, no. 4, ss. 600–612, 2004.
DOI: https://doi.org/10.1109/TIP.2003.819861

[4] K. Zuiderveld, "Contrast Limited Adaptive Histogram Equalization," içinde *Graphics Gems IV*, Academic Press, ss. 474–485, 1994.

[5] OpenCV Documentation, "Histogram Equalization and CLAHE." [Çevrimiçi].
Erişim adresi: https://docs.opencv.org/4.x/d5/daf/tutorial_py_histogram_equalization.html

[6] TensorFlow Lite Documentation, "Deploying models on mobile and edge devices." [Çevrimiçi].
Erişim adresi: https://www.tensorflow.org/lite